

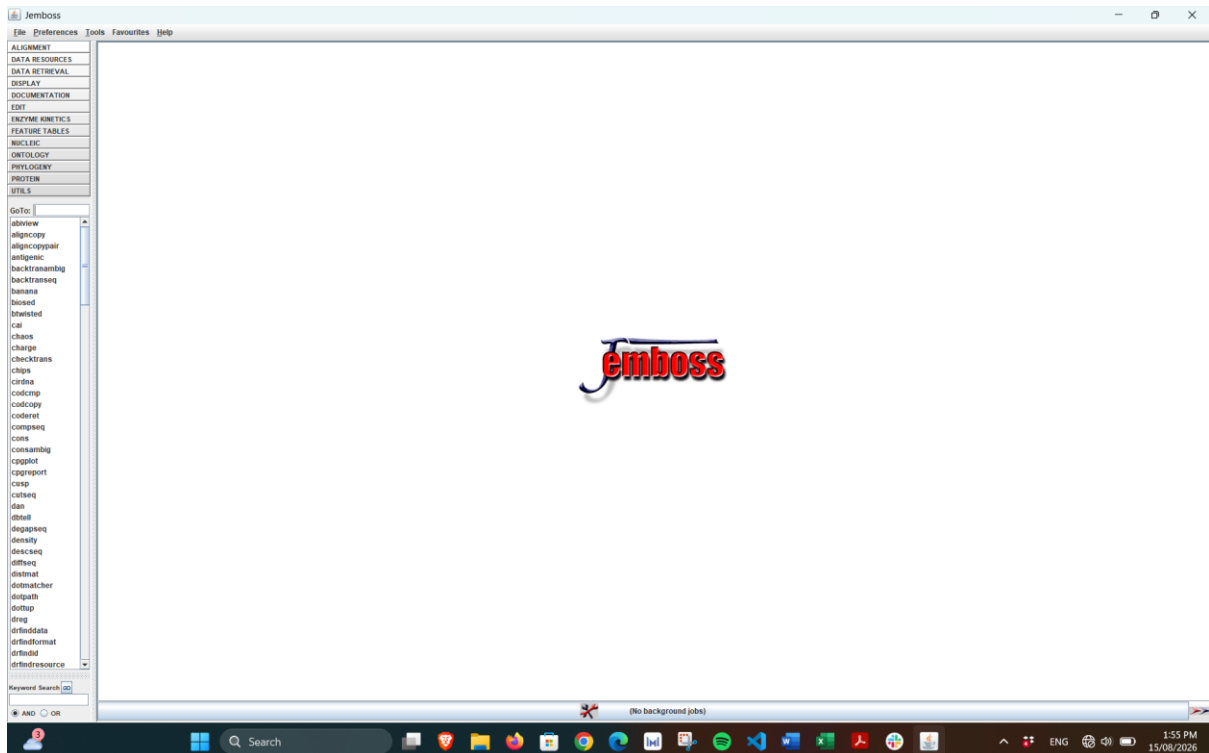
BT3040: Bioinformatics

Assignment 1

- Sidharth Balaji Sridharan
- BS23B008

The code written for each the questions required in this assignment can be found in this github repository. This is currently kept public so everyone can access the code written for each question: [link](#)

Q1) Downloading the EMBOSS Package, Here I've attached the screenshot of the window



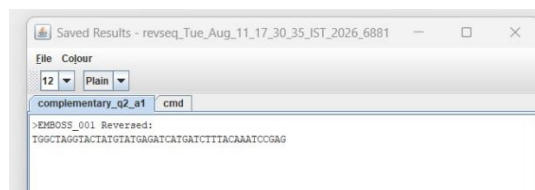
Q2) Finding the Complementary Strand using EMBOSS

The complementary strand was obtained using the revseq feature in EMBOSS, assuming that the direction is 5' – 3', The output of the module is also reported in the 5' – 3' direction.

Output: >EMBOSS_001

Reversed: TGGCTAGGTACTATGTATGAGATCATGATCTTTACAAATCCGAG

Note: The output gives the reversed complementary DNA strand to the given DNA sequence



Q3) Writing code to find the complementary sequence for the given DNA sequence

Note: It isn't mentioned so it is assumed that the direction of the sequence is from 5' – 3' and it's complementary sequence is also expected in the same direction hence I reverse the string

after finding it's complementary sequence.

```
q3.py x q5_a1.py q6_a1.py q8_a1.py seq1.txt
Assignment1 > q3.py > ...
1  dna_code = {
2      "A": 'T',
3      "T": 'A',
4      "G": 'C',
5      "C": 'G',
6  }
7
8
9  def complementary_dna(dna):
10     dna_list = list(dna)
11     for i in range(len(dna)):
12         dna_list[i] = dna_code[dna_list[i]]
13
14     dna = "".join(dna_list)
15     return dna
16
17
18 dna = "CTCGGATTTGTAAAGATCATGATCTCATACATAGTACCTAGCCA"
19 ans = complementary_dna(dna[::-1])
20 print(ans)
```

Output:

```
D:\BT3040\Assignment1>python q3.py
TGGCTAGGTACTATGTATGAGATCATGATCTTTACAAATCCGAG
```

Reverse Complementary Strand:

TGGCTAGGTACTATGTATGAGATCATGATCTTTACAAATCCGAG

(The same exact output the EMBOSS feature)

Q4)

i) The Protein Sequence for the given DNA sequence, there are 6 reading frames hence 6 possible protein sequences. I used the transeq module in EMBOSS and selected the all 6 frames in the advanced options.

Assuming that the sequence is given in the 5' – 3' direction then the 1st ORF's translated protein sequence is from the N-terminus to the C-terminus direction. We have 6 reading frames because there are 3 unique frames from each side starting with the 1st, 2nd and 3rd nucleotide and this is on both sides hence the six ORF's.

We can also use the six pack feature in EMBOSS transeq module to get all the six reading frames for a given sequence.

```

>_1
DIVNSKKVHAMRKEQKRKQKQBSMGSPMDYSPLPIDKHEPEFGPCRRKLDG
>_2
TL*TVKSMQCARSRGRSSAPWALPWTLLCPSTMSLNLVHAEINWVG
>_3
HCEQ*KSPCNAQGAEEAGQAALHGLSHGLLSAHRQA*A*INSMQKKTGWX
>_4
PIQFSSAWTKFRLMLVDGQRRVVHGRAHGALLALLPLLLLAHCMDFFTVHNV
>_5
HPVFFCMDQIQAHACRWAEESFPWESFWSAACPASSAPCALHGLFYCSQCK
>_6
PSSFLHGFNSGSLSMGRGE*SMGEFMERCLPCLFCLSLRIAWTFLFTMS

```

ii) The reading frame equivalent to “PIQFSSAWTKFRLMLVDGQRRVVHGRAHGALLALLPLLLLAHCMDFFTVHNV” is `_4`, which would imply the first reading frame(from the first nucleotide) of the reverse complement strand. This can be verified with the output of question 5 as well and we can know how sequence translated

Q5) This is the code to translate a given DNA sequence into a protein sequence for all the 6 reading frames. We use the existing codon table as a dictionary and traverse the DNA sequence with index 0th, 1st and 2nd position and use the same function on the reverse complementary sequence to get all six ORF's

```

1 codon_table = {
2     'UUU': 'F', 'UUC': 'F', 'UUA': 'L', 'UUG': 'L',
3     'UCU': 'S', 'UCC': 'S', 'UCA': 'S', 'UCG': 'S',
4     'UAU': 'Y', 'UAC': 'Y', 'UAA': '*', 'UAG': '*',
5     'UGU': 'C', 'UGC': 'C', 'UGA': '*', 'UGG': 'W',
6     'CUU': 'L', 'CUC': 'L', 'CUA': 'L', 'CUG': 'L',
7     'CCU': 'P', 'CCC': 'P', 'CCA': 'P', 'CCG': 'P',
8     'CAU': 'H', 'CAC': 'H', 'CAA': 'Q', 'CAG': 'Q',
9     'CGU': 'R', 'CGC': 'R', 'CGA': 'R', 'CGG': 'R',
10    'AUU': 'I', 'AUC': 'I', 'AUA': 'I', 'AUG': 'M',
11    'ACU': 'T', 'ACC': 'T', 'ACA': 'T', 'ACG': 'T',
12    'AAU': 'N', 'AAC': 'N', 'AAA': 'K', 'AAG': 'K',
13    'AGU': 'S', 'AGC': 'S', 'AGA': 'R', 'AGG': 'R',
14    'GUU': 'V', 'GUC': 'V', 'GUA': 'V', 'GUG': 'V',
15    'GCU': 'A', 'GCC': 'A', 'GCA': 'A', 'GCG': 'A',
16    'GAU': 'D', 'GAC': 'D', 'GAA': 'E', 'GAG': 'E',
17    'GGU': 'G', 'GGC': 'G', 'GGA': 'G', 'GGG': 'G'
18 }
19
20 def dna_clean(dna):
21     return dna.replace("T", "U")
22
23 dna_code = {
24     "A": 'T',
25     "T": 'A',
26     "G": 'C',
27     "C": 'G',
28 }
29
30 def complementary_dna(dna):
31     dna_list = list(dna)
32     for i in range(len(dna)):
33         dna_list[i] = dna_code[dna_list[i]]
34
35     dna = "".join(dna_list)
36     return dna[::-1]
37
38 def protein_translation(dna):
39     protein_list = []
40     for j in range(3):
41         protein1 = ""
42         for i in range(j, len(dna), 3):
43             if len(dna[i:i+3]) == 3:
44                 protein1 += codon_table[dna[i:i+3]]
45         protein_list.append(protein1)
46
47     # protein2 = ""
48     # for i in range(len(dna) - j - 3, -1, -3):
49     #     if len(dna[i:i+3]) == 3:
50     #         protein2 += codon_table[dna[i:i+3]]
51     # protein_list.append(protein2)
52
53     return protein_list
54
55
56 dna = "GACATTGTGACAGTAAAAAGTCCATGCAATGCCGANGGACAGAGAGGAAGCA"
57 rna = dna_clean(dna)
58 comp_rna = dna_clean(complementary_dna(dna))
59
60 protein = protein_translation(rna)
61 protein1 = protein_translation(comp_rna)
62
63 print(protein)
64 print(protein1)

```

Output of the code gives the protein sequence of all the 6 reading frames, it's the same exact output when compared to the EMBOSS feature:

1,2,3 : ['DIVNSKKVHAMRKEQKRKQGGKQSRMSGSPMDYSPLPIDKHEPEFGPCRRKLDG',
'TL*TVKKSMQCARSRRGSRASSAPWALPWTTLLCPSTSMSLNLVHAEENWM',
'HCEQ*KSPCNAQGAEEEEAGQAALHGLSHGLLSSAHRQA*A*IWSMQKKTGW']

4, 5, 6: ['PIQFSSAWTKFRLMLVDGQRRVVHGRAHGALLALLPLLLLAHCMDDFFTVHNV',
'PSSFLLHGPNSGSLSMGRGE*SMGEPMERCLPCFLFCSLRIAWTFLFTM',
'HPVFFCMDQIQAHACRWAEESPWESPWSAACPASSAPCALHGLFYCSQC']

Q6) The Code to search substrings of DNA in a given DNA sequence:

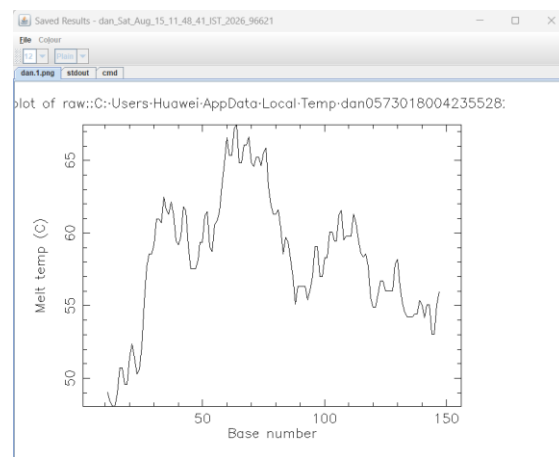
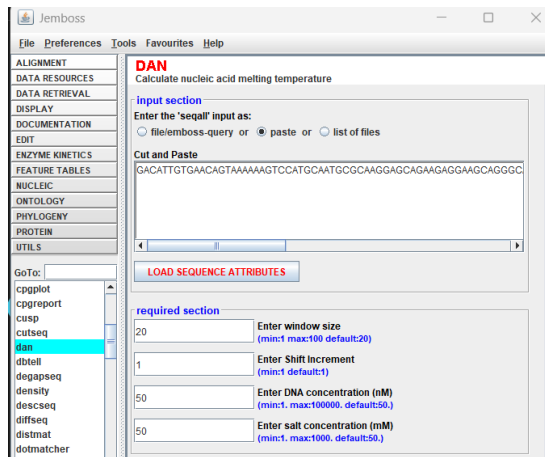
```
q3.py M q5_a1 X q6_a1.py q8_a1.py seq1.txt
Assignment1 > q6_a1.py > ...
1  def matches(dna, subdna):
2      match = 0
3      position = []
4      for i in range(len(dna)):
5          for j in range(len(subdna)):
6              if i + j <= len(dna) - 1 and subdna[j] == dna[i + j]:
7                  if j == len(subdna) - 1:
8                      position.append(i)
9                      match += 1
10                 else:
11                     continue
12             else:
13                 break
14
15     return match, position
16
17 dna = "GACATTGTGAACAGTAAAAAGTCCATGCAATGCGCAAGGAGCAGAAGAGGAAGCAGGGCAAGCAC
18 n_substr = input("Enter the number of strings you want to add:")
19 for i in range(int(n_substr)):
20     subdna = input("Enter a String:")
21     tot_match, pos = matches(dna, subdna)
22     print("Total Matches:", tot_match)
23     print("Position of Match:", pos)
24
25 # subdna = input("Enter a String:")
26 # tot_match, pos = matches(dna, subdna)
27
```

The terminal output for the given set of input strings to check for in a given DNA sequence:

```
D:\BT3040\Assignment1>python q6_a1.py
Enter the number of strings you want to add:5
Enter a String:AAG
Total Matches: 7
Position of Match: [19, 36, 45, 51, 60, 111, 140]
Enter a String:GTC
Total Matches: 2
Position of Match: [21, 130]
Enter a String:GAG
Total Matches: 3
Position of Match: [39, 47, 117]
Enter a String:ACTA
Total Matches: 1
Position of Match: [88]
Enter a String:ATAT
Total Matches: 0
Position of Match: []
```

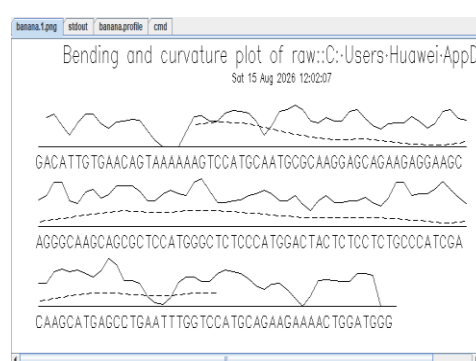
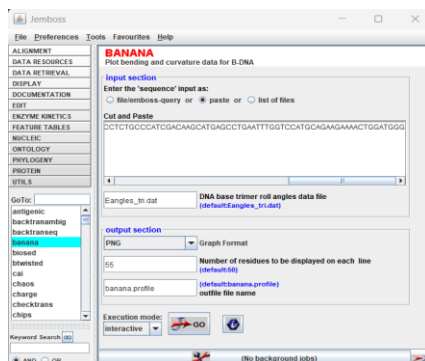
Q7) I was familiarizing myself with the other applications in EMBOSS and explored Melting Temperature, Bending and Curvature

i) Melting Temperature: For this feature, I used DAN



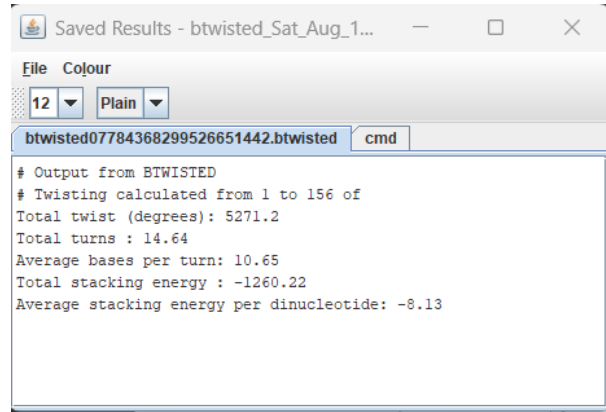
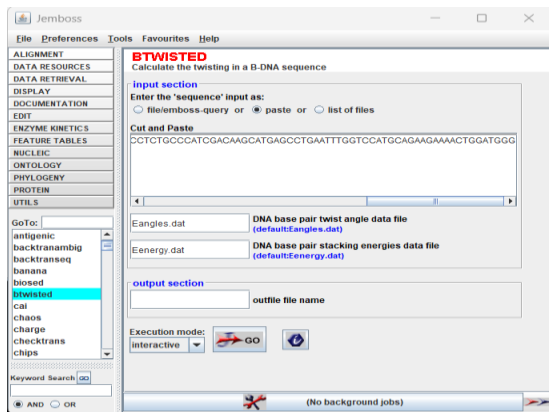
DAN basically outputs the profile of T_m's and not a single overall T_m for the given DNA sequence. The melting temperature here varies from 45° to 70° and the T_m peak occurs around 65-75th base. This is because of it being a GC rich and a thermally stable region and lowered T_m values near the beginning of the sequence and the end because of the AT content in the sequence.

ii) Bending: I used BANANA in EMBOSS to explore this property



The EMBOSS Banana Analysis predicts the intrinsic bending propensity of the DNA sequence and across the entire DNA sequence there's fluctuating bending profile across the DNA. The higher bending values correspond to the AT regions in the DNA and the lower bending values correspond to the GC rich region in the DNA sequence and it also explains because of the lowered degree of freedom because of more H-Bonds for GC nucleotides over AT nucleotides.

iii) Curvature: To analyze and explore curvature, I used the same DNA sequence and used btwisted EMBOSS feature



Looking at the numbers, the average bases per turn is 10.65 and for a B-Form DNA is supposed to be 10.5 roughly. The large negative stacking energy indicates thermodynamic stability between the adjacent nucleotides indicating a stable DNA Helix form for this particular DNA sequence. The Sequence also has a low intrinsic curvature and has a stable helical structure based on the thermodynamic and twist/turn values, indicating that this sequence most likely is a very conventional B-Form DNA which is highly stable. There is also nothing about unusually high curvature as well as it's just 14.64 turns for a 156bp long sequence.

Q8) The Program to compute the average base stacking energy for a given sequence, we essentially use a dictionary for the energy values for every possible consecutive pair and traverse the dna sequence and add the energy values and divide by $\text{len}(\text{dna}) - 1$ because that is the number of pairs for a given DNA sequence

```

q3.py M q5_a1 X q6_a1.py q8_a1.py M seq1.txt
Assignment1 > q8_a1.py > ...
1  avg_base_energy = {
2  |  "AA": -4, "AT": -7, "AC": -5, "AG": -11, "TA": -7, "TT": -2, "TC": -3, "TG": -4, "CA": -9, "CT": -5, "CC": -6, "CG": -7,
3  |  }
4
5  def base_stacking(dna):
6  |  energy = 0
7  |  for i in range(len(dna) - 1):
8  |  |  energy += avg_base_energy[dna[i:i + 2]]
9  |  return energy
10
11 dna = "CTCGGATTGTAAAGATCATGATCTCATACATAGTACCTAGCCA"
12 | total_energy = base_stacking(dna)
13 | stacking_energy = base_stacking(dna)/(len(dna) - 1)
14 | print("Total energy for the given DNA sequence:", total_energy)
15 | print("The average base stacking energy for the given dna sequence is:", stacking_energy)

```

Output for the code:

- i) Total energy for the given DNA sequence: -276
- ii) The average base stacking energy for the given DNA sequence is: -6.4186046511627906

Q9) Using Seq2Feature I computed the average melting temperature and other thermodynamic properties of DNA sequence (i) and (ii)

Input_row	Stacking energy	Enthalpy	Entropy	Flexibility	Flexibility	Free energy	Melting Temperature	Mobility to	Mobility to	Probability	Rise stiffness	Roll stiffness	Shift stiffness	Slide stiffness	Tilt stiffness	Twist stiffness
1	1.62	5.44	14.96	2.277	8.697	0.59	43.202	0.988	0.93	6.08	7.027	17.4	0.803	2.395	25.2	23.3
2	1.58	9.97	24.8	5.842	3.778	1.673	97.08	0.898	1.085	3.04	7.257	19.4	1.032	2.105	28.4	18.1

From this we can explain the results as follows:

i) The GC rich sequence has a higher melting temperature because it is more thermally stable as GC forms 3 Hydrogen bonds which is tougher to break compared to AT which forms two hydrogen bonds

ii) Ergo compared to the AT rich sequence, we would require more energy to break the DNA strand because of the higher number of H-Bonds. The GC strand also has stronger base stacking energy and hence could be another factor for a higher melting temperature

Q10) Using Seq2Feature Locally I computed the AT and the GC content of a given sequence

Input_row	GC_content	Pyrimidine_TC_content	Purine_AG_content	Keto_GT_content	Adenine_content	Guanine_content	Cytosine_content	Thymine_content
1	41.666667	41.666667	58.333333	33.333333	41.666667	16.666667	25	16.666667

AT Content: 58.3333%

GC Content: 41.6667%

The procedure to calculate it would just be to compute the frequency of A/T/G/C in the given sequence. Then GC Content and AT content can be calculated based on the proportion of each frequency.